

Práctica

Presentación

Por fin llegamos al final del curso. Esta práctica servirá para sintetizar todos los conocimientos del curso y ampliarlos con el diseño de circuitos más complejos. En esta práctica se os pedirá el diseño de un circuito a partir de un grafo de estados y el diseño de una máquina de estados en concreto. Para la primera parte, tendréis que aplicar los conocimientos que habéis adquirido en este curso como, por ejemplo, los mapas de Karnaugh o los sistemas de representación de la información. Por lo tanto, se recomienda que repaséis los módulos correspondientes.

Competencias

Conocer la organización general de un computador como circuito digital y conocer los rasgos distintivos de la arquitectura de Von Neumann.

Objetivos

- Conocer varios modelos de las máquinas de estados y de las arquitecturas de controlador con camino de datos.
- Haber adquirido una experiencia básica en la elección del modelo de máquina de estados más adecuado para la resolución de un problema concreto.
- Ser capaz de diseñar circuitos secuenciales a partir de grafos de transiciones de estados.

Recursos

Los recursos que se recomienda usar para esta práctica son los siguientes:

- **Básicos:** El módulo 5 de los materiales.
- **Complementarios:** VerilCIRC, VerilCHART y el Wiki de la asignatura.

Criterios de valoración

- **Razonad la respuesta en todos los ejercicios:** Las respuestas sin justificación no recibirán puntuación.
- **La valoración está indicada en cada uno de los subapartados.**



Uso de herramientas de IA

En esta actividad el uso de herramientas de inteligencia artificial no está permitido. En el plan docente y en el [web sobre integridad académica y plagio](#) de la UOC encontraréis información sobre que se considera conducta irregular en la evaluación y las consecuencias que puede tener.

Formato y fecha de entrega

- Para dudas y aclaraciones sobre el enunciado, diríjós al consultor responsable de vuestra aula.
- Hay que entregar la solución en un fichero PDF que emplee una de las plantillas entregadas juntamente con este enunciado.
- Se tiene que entregar a través de la aplicación de **Entrega de la Actividad** correspondiente del apartado **Contenidos** de vuestra aula.
- La fecha límite de entrega es el **5 de junio**, a las 24 horas.

Ejemplo de solución

PRIMERA PARTE [65%]

Dada la máquina de estados siguiente:

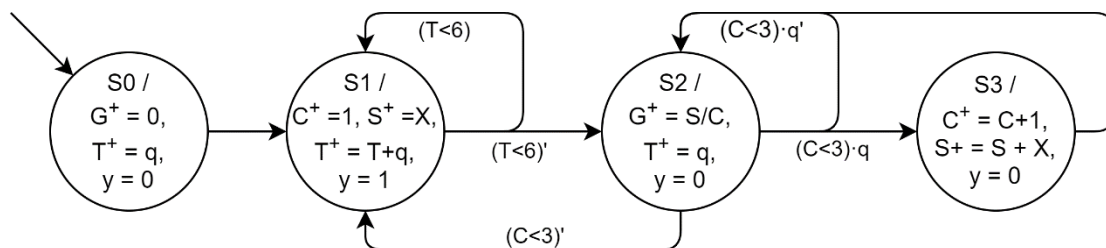


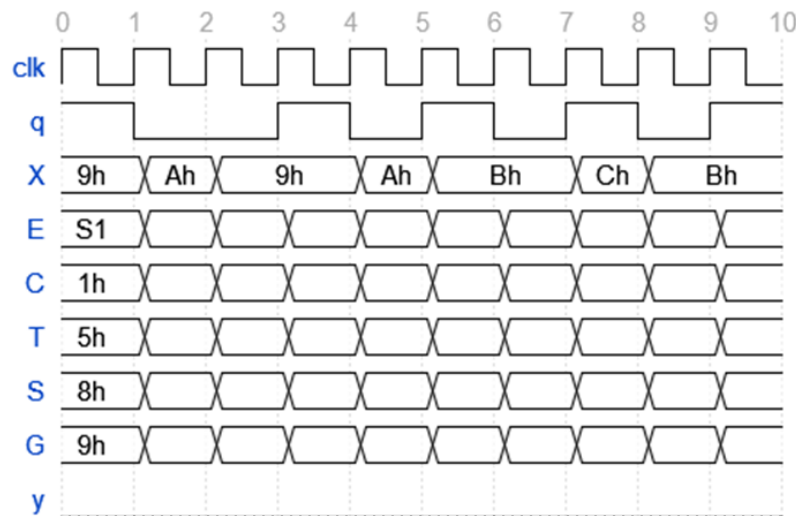
Fig. 1. EFSM de un controlador simplificado de un sensor de glucosa.

Esta máquina de estados tiene, como entradas, una señal q de un bit que indica el paso de una cierta cantidad de tiempo y una señal de datos X del nivel de glucosa en sangre, entre 0 y 31, y, como salidas, una señal binaria de lectura (y) y el valor calculado del nivel de glucosa en sangre G , también entre 0 y 31. Las variables C , S y T son números naturales de 2, 7 y 3 bits, respectivamente. En este sentido, la división S/C es una división entera. Sabiendo todo esto, se pide que:



- a) [15%] A partir del grafo del EFSM de la Fig. 1 completa, en el cronograma siguiente, la evolución del estado (E) y de las señales C , T , S , G e y en cada ciclo de reloj.

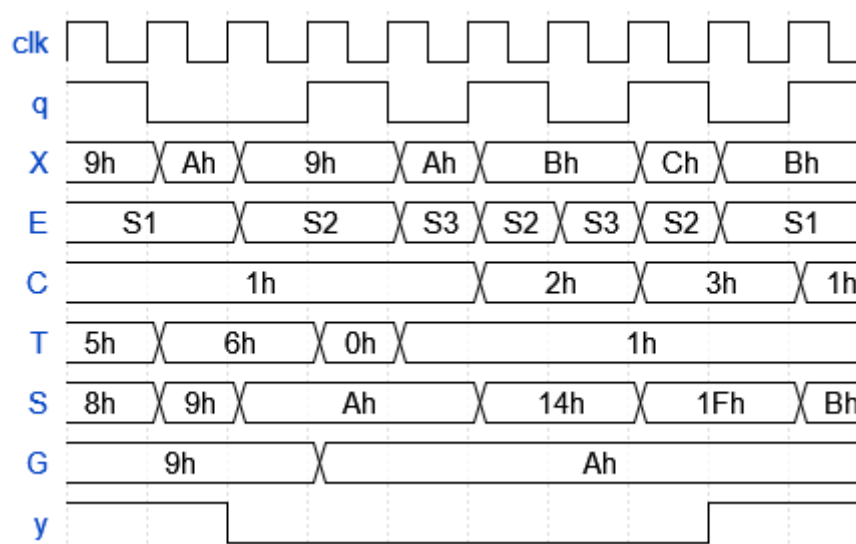
Nota: Tenéis el ejercicio disponible en VerilChart, donde el estado es E y los valores de las señales numéricas se representan en hexadecimal.



De acuerdo con el cronograma, la máquina de estados se encuentra en el estado $S1$ y, como que $T < 6$, el estado siguiente será el mismo $S1$, con la actualización de valores de C , S y T , de los cuales, cambia S porque $X=9h$ y T porque $q=1$. El valor de la señal y depende de forma exclusiva y combinacional del estado en que se encuentra la máquina en cada momento. Dado que la máquina está en el estado $S1$, su valor es 1.

En el ciclo siguiente, como que $T=6h$, la máquina pasa a $S2$ y se actualizan los valores de C , S y T de acuerdo con las expresiones que hay en $S1$. Así pues, pasarán a ser 1 (mantiene el valor), Ah y 6 (el valor no cambia porque se suma 0), respectivamente.

De manera similar se pueden determinar los valores para cada ciclo de reloj a partir de los valores del ciclo anterior y se puede ir repitiendo este procedimiento hasta completar el cronograma, tal como se muestra en la figura siguiente.





- b) [15%] Obtén las tablas de transiciones y de salidas de la EFSM de la Fig. 1, así como las codificaciones binarias de estados y salidas correspondientes. Para simplificar la notación, ($T < 6$) y ($C < 3$) se denominarán $T6$ y $C3$, respectivamente.

En el grafo se observa que las señales de entrada q , $T6$ y $C3$ condicionan las transiciones. También se ve que la entrada X solo se utiliza en el cálculo del valor futuro de la variable S . Para las variables G , T , C y S hay que diferenciar los diversos cálculos y valores que pueden tener, del mismo modo que hay que determinar los valores de la salida y según el estado.

Las tablas de transiciones y salidas que se obtienen a partir del grafo son:

Estado actual	Entrada	Estado siguiente	Estado	Salidas
S0	--	S1	S0	$y=0, G^+ = 0, T^+ = q$
S1	$T6'$	S2	S1	$y=1, C^+ = 1, T^+ = T + q, S^+ = X$
S1	$T6$	S1	S2	$y=0, G^+ = S/C, T^+ = q$
S2	$C3'$	S1	S3	$y=0, C^+ = C + 1, S^+ = S + X$
S2	$C3 \cdot q'$	S2		
S2	$C3 \cdot q$	S3		
S3	--	S2		

Para la codificación binaria de los estados hay que asignar a cada uno un código binario individual. En este caso, se sigue el criterio más habitual, que es el de la numeración binaria, desde el 00 ($S0$) hasta el 11 ($S3$).

Como que todas las variables tienen 3 posibles valores a elegir, si se incluye el hecho de mantener el valor, hacen falta selectores de dos bits. En este caso, se puede hacer coincidir con el código del estado, de forma que:

Estado	Selector	G^+	T^+	C^+	S^+
S0	00	0	q	C	R
S1	01	G	$T + q$	1	X
S2	10	S/C	q	C	S
S3	11	G	T	$C + 1$	$S + X$

El valor de la salida y se determina directamente a partir del código del estado, de forma que $y = (E=S1)$.



Finalmente, las tablas de transiciones y de salidas con la codificación binaria correspondiente son:

Estado actual		Entrada			Estado ⁺
Símbolo	q_1q_0	T6	C3	q	$q_1^+q_0^+$
S0	00	x	x	x	01
S1	01	0	x	x	10
S1	01	1	x	x	01
S2	10	x	0	x	01
S2	10	x	1	0	10
S2	10	x	1	1	11
S3	11	x	x	x	10

Estado			Selector
Símbolo	q_1q_0	y	s_1s_0
S0	00	0	00
S1	01	1	01
S2	10	0	10
S3	11	0	11

Hay que tener en cuenta que los bits de selección son los mismos del estado.

Un filtro de media de una dimensión proporciona, como medida de una señal en un determinado instante de tiempo, el valor de la media de las últimas medidas que ha tomado de la señal. Para este ejercicio, se tendrá que implementar una versión simplificada que trabaje con ventanas de 8 valores de la señal de entrada X y con datos enteros positivos de 0 a 7. Así pues, la señal de salida Y será inicialmente 0 y, después, un valor entre 0 y 7 correspondiente a una aproximación de la media de los 8 últimos valores de X, tal como se muestra en la Fig. 2.

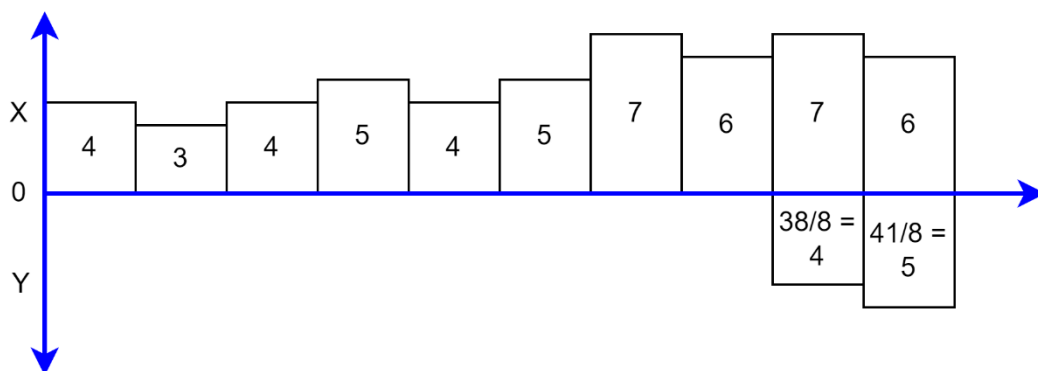


Fig. 2. Evolución de la señal X y de la media simplificada Y

Hay que tener en cuenta que las divisiones son enteras. Por ejemplo, $38/8 = 4$ y no 4,75.



El comportamiento de este filtro se representa en el diagrama de estados de la Fig. 3.

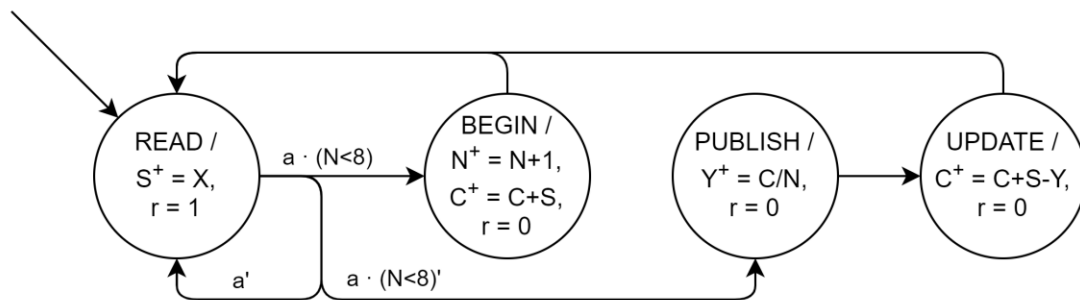


Fig. 3. EFSM de un filtro de media simplificado

En este modelo se ha añadido, a la salida Y , que proporciona la media de los 8 últimos valores, la salida r , que indica si la máquina está preparada para recibir un nuevo dato.

A la vista de la EFSM correspondiente, se pide que:

- c) [15%] A partir de las tablas de transiciones y de salidas del grafo de la Fig. 3 que se dan a continuación, implementa la unidad de control usando ROM y los registros, los bloques combinacionales y las puertas lógicas que creas conveniente. Especifica y justifica las dimensiones de la/las ROM que uses e indica su contenido en binario, indicando a que corresponde cada bit, y también en hexadecimal.

Estado actual	Entradas de control		Estado*
	a	(N<8)	
READ	0	x	READ
READ	1	0	PUBLISH
READ	1	1	BEGIN
BEGIN	x	x	READ
PUBLISH	x	x	UPDATE
UPDATE	x	x	READ

Estado		Salidas				
Símbolo	q_1q_0	q	s_N	s_C	s_Y	s_S
READ	00	1	0	00	0	1
BEGIN	01	0	1	01	0	0
PUBLISH	10	0	0	00	1	0
UPDATE	11	0	0	10	0	0

Nota: El ejercicio está disponible en VerilCIRC y, si quieres comprobar previamente la corrección de una ROM válida con palabras de 8 bits, está disponible en VerilChart. Ten en cuenta que hay otras opciones igualmente válidas con ROM de diferentes medidas. En este entorno, el nombre de la señal de estado es E y el de la entrada de control $(N<8)$ es $N8$.

Para implementar la unidad de control con una ROM hace falta, primero, determinar su tamaño y, después, calcular su contenido a partir de la tabla de transiciones correspondiente.



El bus de direcciones de la ROM tiene que ser de 4 bits, puesto que la dirección se forma con los bits que codifican el estado (en este caso son 2) y los de las señales de entrada (2 más). Por lo tanto, la ROM tiene que disponer de un total de $2^4 = 16$ posiciones de memoria.

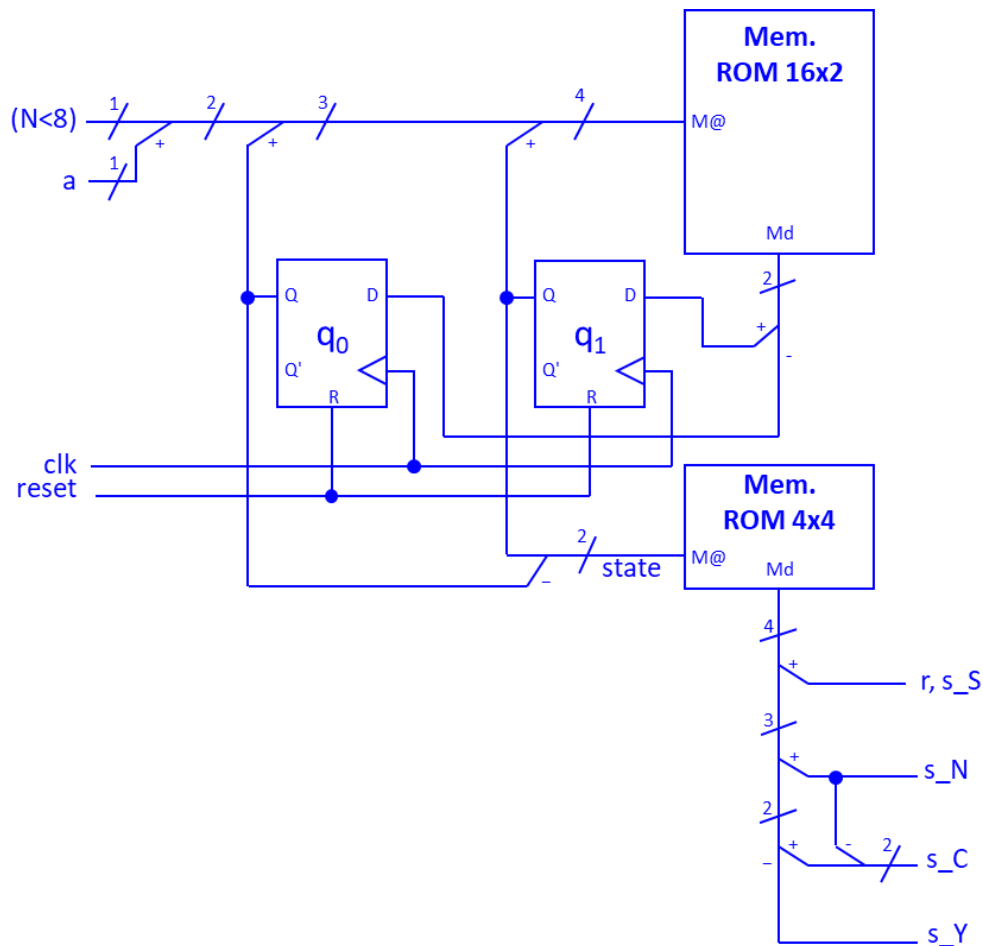
En cada posición se almacena el código del estado siguiente y el valor, en el estado actual, de cada una de las señales de salida. Por lo tanto, hacen falta 2 bits para el estado y harían falta 6 bits para las salidas (r , s_N , s_C , s_Y , y s_S), cosa que daría un total de 8 bits.

Dado que el bit 0 del selector s_C y el selector s_S coinciden con s_N y r , respectivamente, no es necesario que se almacenen en la ROM.

Así pues, si se usa una única ROM, tendrá que ser de 16x6 bits. El contenido de la ROM se determina a partir de la tabla de transiciones y la de salidas, sin s_C_0 y s_S :

Posición de memoria			Contenido					Codificación hexadecimal
q_1q_0	a	(N<8)	$q_1^+q_0^+$	q	s_N	s_C_1	s_Y	
00	0	0	00	1	0	0	0	08h
00	0	1	00	1	0	0	0	08h
00	1	0	10	1	0	0	0	28h
00	1	1	01	1	0	0	0	18h
01	0	0	00	0	1	0	0	04h
01	0	1	00	0	1	0	0	04h
01	1	0	00	0	1	0	0	04h
01	1	1	00	0	1	0	0	04h
10	0	0	11	0	0	0	1	31h
10	0	1	11	0	0	0	1	31h
10	1	0	11	0	0	0	1	31h
10	1	1	11	0	0	0	1	31h
11	0	0	00	0	0	1	0	02h
11	0	1	00	0	0	1	0	02h
11	1	0	00	0	0	1	0	02h
11	1	1	00	0	0	1	0	02h

Posición de memoria	Contenido				Codificación hexadecimal
q ₁ q ₀	q	s_N	s_C ₁	s_Y	
00	1	0	0	0	8 h
01	0	1	0	0	4 h
10	0	0	0	1	1 h
11	0	0	1	0	2 h



A la vista de los contenidos de la ROM de salidas, también se podría sustituir por puertas lógicas, pero queda fuera de lo que se pide en el ejercicio.

- d) [20%] Implementa el circuito completo de la EFSM de la Fig. 3: Construye el camino de datos usando las puertas lógicas y los bloques necesarios e incorpora la unidad de control obtenida en el apartado anterior. Ten en cuenta que la señal de entrada X , S y la salida Y son números naturales de 0 a 7. Indica y razona, para cada bus, su tamaño en bits. Ten en cuenta también que, con la inicialización del circuito, todos los registros se ponen a 0.

Nota: El ejercicio está disponible en VerilCIRC. En el circuito, las señales N , C y S también son de salida para poder comprobar el funcionamiento más fácilmente. Ten en cuenta que VerilCIRC no puede verificar subcircuitos diseñados por el usuario y, por lo tanto, tendrás que copiar en tu solución el circuito de la unidad de control que hayas diseñado en el apartado anterior.

Por un lado, el camino de datos se tiene que ocupar de generar la señal $N8$ para la unidad de control. Por el otro, tiene que hacer todos los cálculos posibles de los nuevos valores de las variables N , C , Y y S , y almacenarlos en los registros correspondientes según las señales de control que correspondan. En este caso, además, hay una señal de salida inmediata (r), sin que se tenga que almacenar en ninguna parte.



La dimensión del bus que transporta la señal X y de los registros que almacenan S y Y tiene que ser de 3 bits, puesto que tienen que poder representar del 0 al 7.

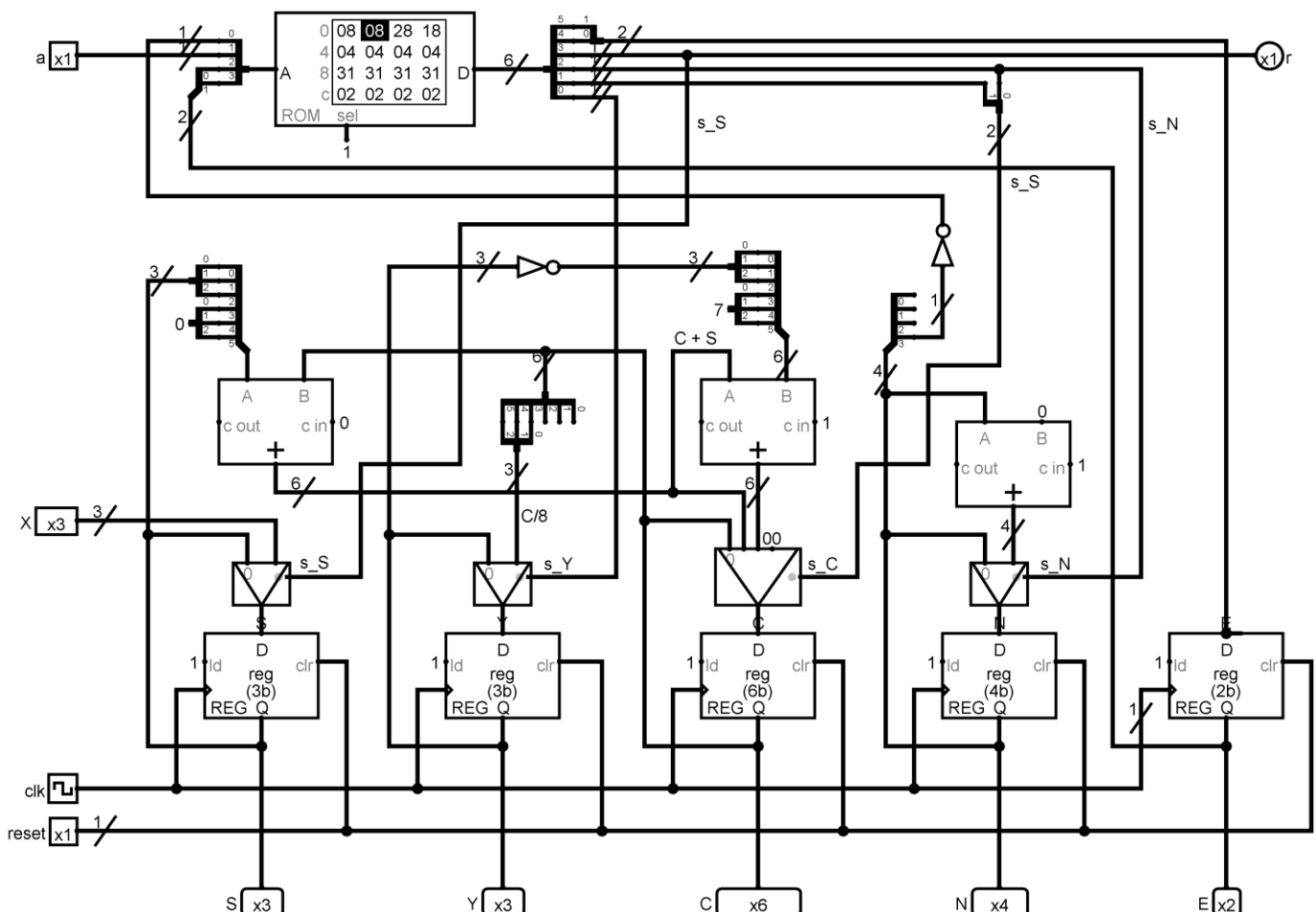
Dado que N tiene que llegar a 8, el registro correspondiente tendrá que ser de 4 bits. De hecho, la señal $N8$ será el complemento de su bit más significativo, n_3 , es decir, cuando $N < 8$, $n_3 = 0$ y $N8 = 1$.

En el caso de C , que acumula hasta 8 valores de X , tendrá que poder representar desde el número 0 hasta el $7 \cdot 8 = 56$ y, por lo tanto, tendrá que ser de 6 bits.

Los selectores s_N , s_Y y s_S son de un único bit, 0 para mantener el valor y 1 para cambiarlo.

Dado que a C se le puede asignar C , $C+S$ y $C+S-Y$, su selector de valor para el ciclo siguiente es de 2 bits, con 00 para mantener el valor y 01 y 10 para almacenar el resultado de las operaciones $C+S$ o $C+S-Y$, respectivamente.

Hay varias maneras de implementar $C+S$ y $C+S-Y$. En la propuesta que se muestra a continuación se ha optado por calcular $C+S$ y, entonces calcular $C+S-Y$ como una resta entre $C+S$ e Y , de forma que se aprovecha el sumador de C y S . Eso sí, esto es posible porque, como mucho, $C+S = 56+7 = 63$, que también se puede representar con 6 bits. Con todo esto, el circuito completo para el EFSM de la Fig. 3 queda de la forma siguiente:





SEGUNDA PARTE [35%]

Se tiene que diseñar el modelo de un módulo de multiplicación de matrices (MMM) que usa dos memorias: una (M0) que contiene la primera matriz (A) y la otra (M1) que almacena la segunda matriz (B) y la matriz resultante (C). Este tipo de bloques son muy utilizados tanto en el procesamiento de todo tipo de señales y de imágenes como para los cálculos de las redes neurales en aplicaciones de inteligencia artificial.

El MMM tiene que implementar el comportamiento que se describe en el programa en C siguiente:

```
int N = 3; // N>0!
int M0[] = {1, 2, 3, 4, 5, 6, 7, 8, 9};
int M1[] = {1, 2, 3, 4, 5, 6, 7, 8, 9,
            0, 0, 0, 0, 0, 0, 0, 0, 0};

int BA = 0; // base of A
int BB = 0; // base of B
int AA; // address at A
int AB; // address at B
int AC = N*N; // address at C
int MAC = 2*N*N; // max address at C
int MAA; // max address at A
int P; // sum of products
do { // for each element in C
    P = 0; AA = BA; AB = BB; MAA = BA + N;
    do { // sum of products
        P = P + M0[AA]*M1[AB];
        AA = AA + 1; AB = AB + N;
    } while(AA<MAA);
    M1[AC] = P;
    AC = AC + 1;
    BB = BB + 1; if(BB>=N) {BB = 0; BA = BA + N;}
} while(AC<MAC);
```

El tamaño de las matrices, N , viene dado por una señal de entrada y, por lo tanto, es un valor variable que, en el ejemplo, es 3. Las memorias M0 y M1 tienen el tamaño necesario para contener las matrices A y B y C, respectivamente. Los valores de las matrices son tales que no hay que preocuparse por ningún desbordamiento y el módulo que hace el producto ajusta el resultado al mismo formato que los multiplicandos. (En la realidad, estas multiplicaciones siempre se hacen con matrices *normalizadas*, es decir, que cumplen esta propiedad, entre otras.)

En las memorias, las matrices se almacenan por filas y columnas. El ejemplo anterior sería:

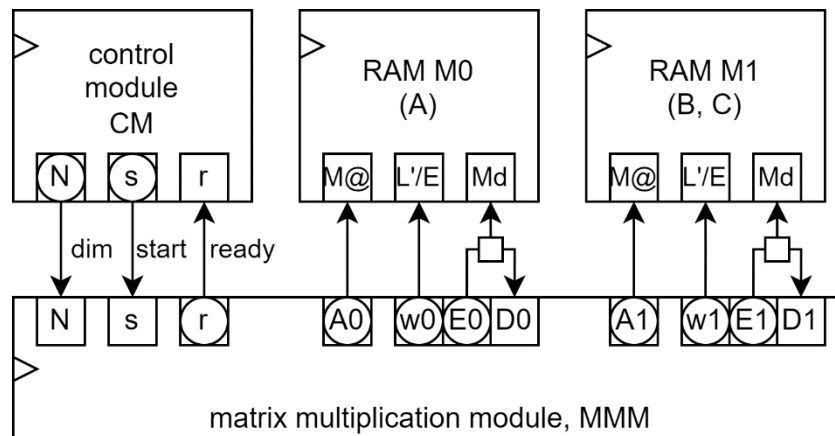
$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix} \times \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix} = \begin{pmatrix} 30 & 36 & 42 \\ 66 & 81 & 96 \\ 102 & 126 & 150 \end{pmatrix}$$

A B C



El modelo que se tiene que diseñar debe tener en cuenta la comunicación con las memorias M0 y M1, y con el módulo CM, que indica cuando tiene que iniciar una multiplicación y cuál es el tamaño de las matrices, N .

El módulo tendrá que empezar a hacer la multiplicación en el ciclo en que la señal de inicio s (de *start*) se active, es decir, cuando $s=1$. Mientras esté haciendo los cálculos tendrá que poner la señal de indicación de disponibilidad r (de *ready*) a 0 y dejarla en 1 en acabar.



La comunicación con las memorias se hace con las salidas que contienen las direcciones de memoria ($A0$, $A1$), las señales de salida ($w0$, $w1$) que indican qué tipo de operación (lectura o escritura) se hace en ellas, las salidas que contienen los datos a escribir ($E0$, $E1$) y las entradas que proporcionan lo que se lee en las memorias ($D0$, $D1$). En el caso de M0, $w0$ siempre será 0 y $E0$ se puede dejar también a 0, puesto que no se harán escrituras en M0. En el caso de M1, en el ciclo que $w1=1$, se escribirá lo que haya en $E1$ y $D1=0$.

Teniendo en cuenta que todas las salidas del modelo del MMM son variables del mismo modelo, diseñad el ASM correspondiente.

El diagrama de estados del ASM se puede hacer siguiendo el programa de ejemplo, empezando por una inicialización de las variables del programa, que también lo serán de la máquina de estados. A esta inicialización se le tiene que añadir la de las variables de relación con las memorias y con el módulo de control. En este sentido, la variable del programa que contiene la dirección de la matriz A (AA) es la misma que $A0$, puesto que M0 solo contiene A.

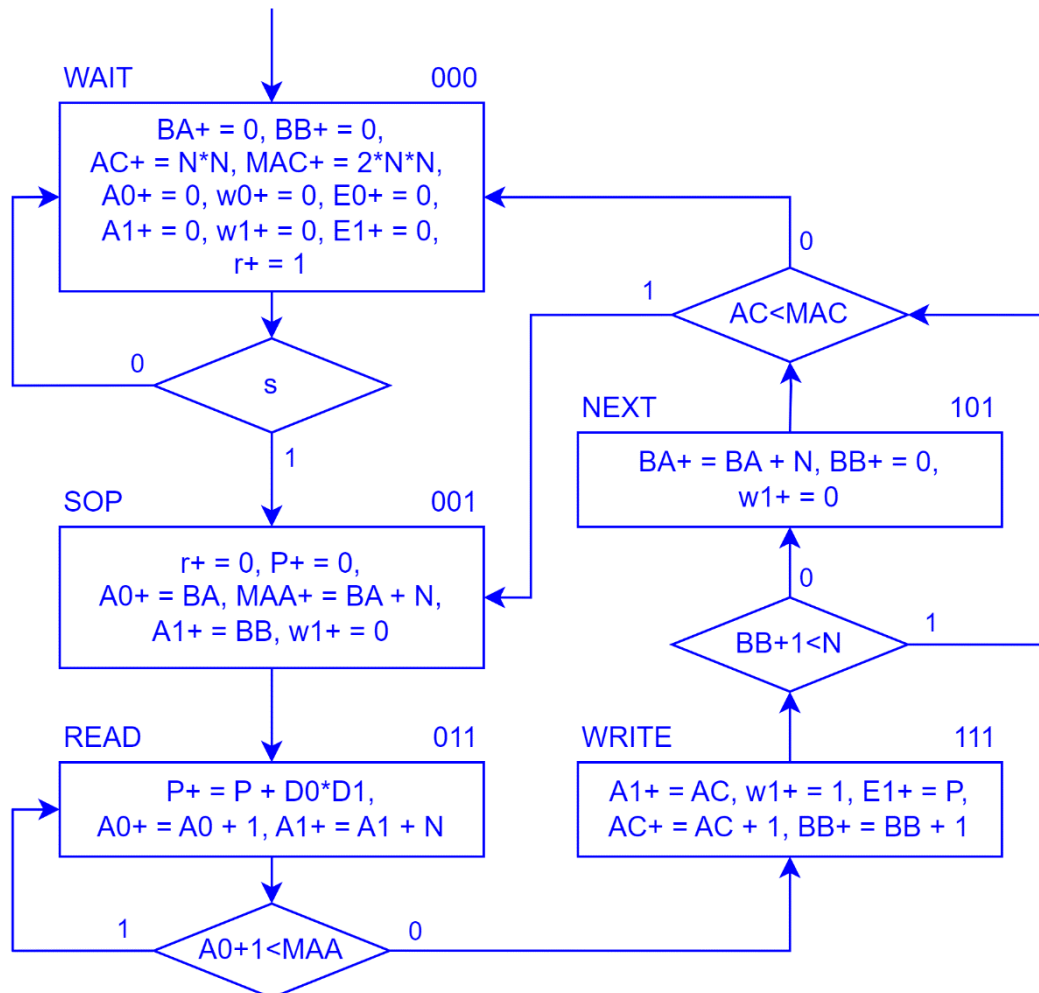
La máquina tendrá que estar en este primer estado (WAIT) hasta que $s=1$. En este momento, pasará a SOP, que prepara las variables para calcular una suma de productos para un elemento de C. En este estado también se pondrá r a 0, para indicar al CM que está ocupado haciendo cálculos. De forma similar a como se ha hecho con la variable del programa AA , aquí AB se sustituye directamente por $A1$, puesto que la suma de productos se acumula a P y no se hace ninguna escritura a M1.

De SOP pasa a READ para hacer el cálculo de un producto y cambiar los valores de las direcciones de M0 y de M1. En READ se comprueba que la nueva dirección no haya llegado al máximo para hacer un nuevo producto y acumularlo a P. Si ya ha acabado, pasa a WRITE.



En el estado WRITE se prepara la escritura de P y se comprueba que no se tenga que cambiar de columna de B . Si es el caso, pasa a NEXT. Dado que en WRITE se ha puesto $w1$ a 1, en todos los estados de salida (NEXT y SOP y WAIT) se tiene que poner $w1$ a 0, de forma que, en caso de tener que continuar con los cálculos, se continúen haciendo lecturas a $M1$.

La figura siguiente muestra la ASM correspondiente a este comportamiento:



Hay que tener presente que la asignación $w1^+ = 0$ que hay en NEXT no es estrictamente necesaria y que también se puede resolver usando variables AA y AB equivalentes a las que hay en el programa de ejemplo y asignarlas a $A0$ y $A1$ cuando convenga, aunque la solución necesite de estados adicionales.